

Org Admin Page Handoff Guide

Updated: 2026-03-26

Audience: product, design, QA, support, and engineering

Repository state basis: current repo on 2026-03-26, the active implementation in `src/org/admin/*`, related Supabase migrations/functions, and the in-repo org/admin implementation notes and audits

1. Purpose of This Document

This document is the complete handoff guide for the Org Admin Page project.

It is written to do two jobs at once:

1. Explain the product in plain language to non-technical stakeholders.
2. Give a new developer enough technical depth to recreate, maintain, and extend the current system without relying on tribal knowledge.

In this codebase, "Org Admin Page" refers to the organization-scoped admin control plane mounted inside the org workspace at:

```
? /app/org/:orgId/admin/members
? /app/org/:orgId/admin/roles
? /app/org/:orgId/admin/pipelines
? /app/org/:orgId/admin/credits
? /app/org/:orgId/admin/settings
```

This is different from the platform-wide admin dashboard under `/app/admin/*`.

The platform admin dashboard is still relevant because it provisions organizations and kicks off the owner invite/bootstrap flow that makes the org admin pages usable.

Table of Contents

1. Purpose of This Document
2. Executive Overview
3. Product Mental Model
4. Route Map
5. History and Evolution
6. Current State Summary
7. High-Level Architecture
8. Code Structure
9. Data Model and Database Schema
10. Permission Model
11. RLS and Security Model
12. Org Bootstrap and Provisioning

13. Page-by-Page Deep Dive
14. Service Layer Deep Dive
15. User Flows
16. Screen Guide
17. Challenges, Corrections, and Design Decisions
18. Known Gaps and Risks
19. Step-by-Step Rebuild / Setup Guide
20. Recreating the System Exactly
21. Best Practices for Future Engineers
22. Recommended Reading Order for a New Developer
23. Final Take

2. Executive Overview

What the Org Admin Page is

The Org Admin Page is the operational control surface for a single organization inside the Social Media Agent product.

It allows an organization owner or org admin to manage:

- ? who belongs to the organization
- ? what permissions those members have
- ? how review pipelines work
- ? how credits are observed and requested
- ? a small set of org-wide settings

Why it exists

The rest of the org workspace is the day-to-day execution layer:

- ? My Office for draft creation
- ? Pipeline for review
- ? Calendar for scheduling
- ? Asset Library for reusable assets
- ? Common Room for collaboration

The Org Admin Page is the policy and configuration layer behind those surfaces.

Without it, the product would have no credible way to:

- ? define roles and permission bundles
- ? restrict brand-project access
- ? choose who can publish, schedule, approve, invite, or manage channels/assets

- ? configure the approval workflow that content enters from My Office
- ? monitor and govern credits at the organization level

Core functionality

The current org admin surface is made of five functional pages:

1. Members

Manages membership, invitations, assigned roles, brand-project scope, permission overrides, and per-member credit limits.

2. Roles

Manages org role templates and the permission bundles that members inherit from them.

3. Pipelines

Manages workflow templates used when content is submitted into the org review process.

4. Credits

Displays credit usage and credit requests. This page is thinner than the others and is more monitoring-oriented than action-oriented in the current implementation.

5. Settings

Displays a small set of organization-level settings. This page is intentionally minimal today.

3. Product Mental Model

Non-technical mental model

The org admin pages answer these questions:

- ? Who is allowed into the workspace?
- ? What can each person do?
- ? Which brand projects can they touch?
- ? What review steps does content go through?
- ? How do credits get tracked and requested?
- ? What are the default rules for this org?

Relationship to the wider org workspace

The org admin pages are not isolated.

They determine how the rest of the org workspace behaves.

Examples:

- ? Members and Roles determine who can publish, schedule, manage the library, create channels, or invite others.
- ? Pipelines determine what happens when a draft is submitted from My Office.

? Credits matters for AI-backed org features and credit request workflows.

? Settings influences org-level behavior such as preferred defaults stored in `organizations.settings`.

4. Route Map

The active org admin routes live in `src/router/router.jsx`.

Route	Purpose	Main file
<code>`/app/org/:orgId/admin/members`</code>	member directory, invite flow, role/permission editing	<code>`src/org/admin/MembersPage.jsx`</code>
<code>`/app/org/:orgId/admin/roles`</code>	role-template CRUD and permission bundle editing	<code>`src/org/admin/RolesPage.jsx`</code>
<code>`/app/org/:orgId/admin/pipelines`</code>	pipeline template builder and defaults	<code>`src/org/admin/PipelineConfigPage.jsx`</code>
<code>`/app/org/:orgId/admin/credits`</code>	credit summary and request list	<code>`src/org/admin/CreditManagementPage.jsx`</code>
<code>`/app/org/:orgId/admin/settings`</code>	org metadata and settings summary	<code>`src/org/admin/OrgSettingsPage.jsx`</code>

These routes are wrapped by `OrgAdminRoute` in `src/utils/protectedRoute.jsx`.

Access rules

To reach these routes, the user must:

- ? be authenticated
- ? belong to the target organization
- ? satisfy the org-admin access check (`isOrgAdmin`)

In practice that means the surface is intended for:

- ? `org_owner`
- ? `org_admin`

The sidebar only shows the admin entries when the org context reports `isOrgAdmin`.

5. History and Evolution

This project did not appear all at once.

It grew in stages, and some current design choices only make sense when you understand that history.

Phase 0: Platform admin foundation

Before the current org workspace model matured, the repo already had a platform-admin concept.

That older work introduced:

- ? organizations
- ? admin roles
- ? audit logs

- ? complaints and moderation support
- ? admin shell routes under /app/admin/ *

Key milestone:

- ? 2026-03-12

supabase/migrations/20260312153000_admin_foundation.sql

This created the first governance tables and admin helper functions.

At this point, the system still mixed platform-admin concerns and org membership concepts more heavily than it does now.

Phase 1: Admin recursion and reliability fixes

Shortly after the admin foundation shipped, a recursion issue in RLS/admin helpers had to be corrected.

Key milestone:

- ? 2026-03-13

supabase/migrations/20260313090000_admin_rls_recursion_hotfix.sql

This moved important helper logic to safer SECURITY DEFINER patterns so admin access checks stopped recursively querying protected tables during policy evaluation.

Phase 2: Org workspace foundation

The project shifted from "generic admin tooling" to a true org workspace model.

This is where the current org admin pages really became meaningful.

Key milestones on 2026-03-24:

- ? 20260324100000_org_workspace_foundation.sql
- ? 20260324110000_org_pipeline_tables.sql
- ? 20260324140000_org_credit_tables.sql
- ? 20260324160000_org_rls_policies.sql
- ? 20260324170000_org_helper_functions.sql
- ? 20260324180000_org_seed_plan_data.sql
- ? 20260324190000_org_invitation_owner_provisioning.sql

This evolution added:

- ? organization_plans
- ? brand_projects
- ? richer organization_members
- ? org_role_templates
- ? org_invitations
- ? pipeline configs and items

- ? credit request tables

- ? org-specific helper functions and RLS

This is the real birth of the current org admin pages as a proper control plane rather than a thin wrapper around generic admin tables.

Phase 3: UI maturity and operational corrections

As the broader org workspace matured, the admin area received targeted corrections rather than a total rewrite.

Documented improvements include:

- ? invite flow upgraded to use org role templates, not a bare select

- ? permission preview in the invite panel

- ? better long-table / long-form scrolling in members and roles screens

- ? owner invitation provisioning improvements

Relevant references:

- ? docs/admin-dashboard-implementation-summary-2026-03-12.md

- ? docs/admin-dashboard-corrections-v2-2026-03-13.md

- ? docs/admin-dashboard-audit-2026-03-23.md

- ? docs/admin-dashboard-database-relationship-audit-2026-03-23.md

- ? docs/org-calendar-implementation-report.md

Phase 4: Relationship to later workspace phases

Later work refreshed My Office, Common Room, and Asset Library.

Those later pages depend on the admin control plane but do not replace it.

The org admin pages remained the source of truth for:

- ? role templates

- ? permission modeling

- ? review pipeline configuration

- ? invite and membership governance

The org admin visual language also became the baseline dark theme that later workspace pages were aligned to.

6. Current State Summary

What is mature

- ? org member management

- ? invitation flow

- ? role template modeling

- ? permission inheritance plus overrides
- ? pipeline config management
- ? org-scoped RLS and helper functions

What is present but thinner

- ? credits page
- ? settings page

What is structurally important but still imperfect

- ? the overall org data model still carries legacy compatibility layers from older admin iterations
- ? some governance logic still lives in direct client-side Supabase mutations rather than edge functions

7. High-Level Architecture

The Org Admin Page sits inside the org workspace shell.

Frontend architecture

Main layers:

1. Routing and access control
2. Org context loading
3. Page-level admin UIs
4. Shared service layer
5. Supabase database / RLS / functions

Technology stack

The current implementation is built on:

- ? React 18
- ? Vite 7
- ? React Router 6
- ? Supabase JS v2
- ? Supabase Auth / Postgres / Realtime / Edge Functions
- ? Zustand for some workspace runtime state
- ? React Hot Toast for feedback
- ? Lucide React for iconography

Core frontend files

Shared org shell and runtime:

- ? `src/layouts/OrgWorkspaceShell.jsx`

```
? src/org/context/OrgContextProvider.jsx
? src/org/hooks/useOrgContext.js
? src/org/components/OrgSidebar.jsx
? src/org/components/OrgTopNavbar.jsx
? src/org/stores/orgRuntimeStore.js
```

Admin pages:

```
? src/org/admin/MembersPage.jsx
? src/org/admin/RolesPage.jsx
? src/org/admin/PipelineConfigPage.jsx
? src/org/admin/CreditManagementPage.jsx
? src/org/admin/OrgSettingsPage.jsx
? src/org/styles/OrgAdmin.css
```

Shared org admin components and services:

```
? src/org/components/InviteMemberPanel.jsx
? src/org/services/orgService.js
? src/org/services/pipelineService.js
? src/org/services/creditService.js
? src/org/hooks/useOrgCredits.js
? src/org/constants/permissions.js
```

Platform-admin provisioning layer that creates organizations:

```
? src/admin/components/CreateOrgPanel.jsx
? src/admin/services/orgAdminService.js
```

Edge/backend:

```
? supabase/functions/org-invite-member/index.ts
? supabase/functions/credit-request-action/index.ts
? supabase/functions/_shared/org.ts
? supabase/functions/_shared/org-bootstrap.ts
```

Architecture diagram

```
Platform Admin Create Org
|
v
org-invite-member edge function
|
v
ensureOrganizationBootstrap()
|
+--> organizations
+--> organization_members
+--> org_role_templates
+--> brand_projects
```



```

    +--> pipeline_configs
    +--> org_invitations

User enters /app/org/:orgId/admin/*
    |
    v
OrgAdminRoute
    |
    v
OrgContextProvider
    |
    +--> active organization
    +--> membership
    +--> effective permissions
    +--> brand projects
    +--> active brand project
    |
    v
Org Admin Page (Members / Roles / Pipelines / Credits / Settings)
    |
    v
orgService / pipelineService / creditService
    |
    v
Supabase tables + RLS + helper functions

```

8. Code Structure

The smallest useful code map for a new developer is:

```

src/
  layouts/
    OrgWorkspaceShell.jsx
  router/
    router.jsx
  utils/
    protectedRoute.jsx
  org/
    admin/
      MembersPage.jsx
      RolesPage.jsx
      PipelineConfigPage.jsx
      CreditManagementPage.jsx
      OrgSettingsPage.jsx
    components/
      InviteMemberPanel.jsx
      OrgSidebar.jsx
      OrgTopNavbar.jsx
    constants/
      permissions.js
    context/
      OrgContextProvider.jsx
    hooks/
      useOrgContext.js
      useOrgCredits.js
    services/
      orgService.js
      pipelineService.js
      creditService.js
    stores/
      orgRuntimeStore.js
    styles/
      OrgAdmin.css

src/admin/
  components/
    CreateOrgPanel.jsx
  services/
    orgAdminService.js

supabase/

```

```
migrations/  
  20260312153000_admin_foundation.sql  
  20260313090000_admin_rls_recursion_hotfix.sql  
  20260324100000_org_workspace_foundation.sql  
  20260324110000_org_pipeline_tables.sql  
  20260324140000_org_credit_tables.sql  
  20260324160000_org_rls_policies.sql  
  20260324170000_org_helper_functions.sql  
  20260324180000_org_seed_plan_data.sql  
  20260324190000_org_invitation_owner_provisioning.sql  
functions/  
  org-invite-member/  
  credit-request-action/  
  _shared/  
    org.ts  
    org-bootstrap.ts
```

9. Data Model and Database Schema

This section focuses on the tables and functions that directly matter to the org admin pages.

Core organization tables

organizations

The root org workspace record.

Important fields used by org admin:

- ? id
- ? name
- ? slug
- ? plan
- ? plan_key
- ? status
- ? owner_id
- ? owner_user_id
- ? monthly_credit_allocation
- ? credits_used_this_period
- ? settings

organization_plans

Defines plan-level allocations and features.

Important fields:

- ? plan_key
- ? monthly_credit_allocation
- ? features

This is how the repo models capabilities like:

- ? approval pipeline
- ? common room
- ? shared library
- ? brand projects

organization_members

The most important table behind Members.

Important fields:

- ? organization_id
- ? user_id
- ? role
- ? org_role_key
- ? status
- ? permissions
- ? brand_project_ids
- ? credits_used_this_period
- ? monthly_credit_limit_override
- ? last_active_at

This table stores:

- ? membership
- ? org role assignment
- ? permission overrides
- ? brand-project scope
- ? per-member credit usage

brand_projects

Sub-scope inside an organization, especially important for agency workspaces.

Important fields:

- ? id
- ? organization_id
- ? name
- ? slug
- ? status
- ? settings

org_role_templates

The reusable role definitions managed in Roles.

Important fields:

- ? id
- ? organization_id
- ? role_key
- ? name
- ? description
- ? permissions
- ? is_system_role

org_invitations

Tracks pending/accepted/revoked org invites.

Important fields:

- ? id
- ? organization_id
- ? email
- ? role
- ? brand_project_ids
- ? status
- ? invitation_token
- ? expires_at
- ? invited_by
- ? invited_user_id
- ? requires_password_setup
- ? accepted_at

context_last_used

Stores the last org and brand project used by the current user.

This is written by the org context layer and helps restore the workspace context.

Pipeline tables

pipeline_configs

Defines org-specific or brand-specific review flows.

Important fields:

- ? id

? organization_id
? brand_project_id
? name
? description
? is_default
? stages

pipeline_items

Created when content is submitted into review.

Important fields:

? id
? organization_id
? brand_project_id
? post_id
? submitted_by
? current_stage_index
? current_assignee_role
? current_assignee_user_id
? status
? metadata

Credit tables

credit_events

Ledger-like record of usage and adjustments.

Important fields:

? organization_id
? user_id
? event_type
? amount
? metadata
? created_at

credit_requests

Tracks member requests for more credits and the admin review state.

Important fields:

? organization_id
? requested_by
? requested_amount
? reason
? status
? reviewed_by
? reviewed_at

Related downstream tables controlled indirectly by org admin

The admin pages do not own these tables, but their rules influence them:

? posts
? generations
? org_asset_library
? common_room_channels
? common_room_messages

Examples:

? can_manage_library affects the asset library.
? can_create_channels affects Common Room channel management.
? pipeline config selection affects pipeline_items created from My Office.

10. Permission Model

The permission model lives across:

? src/org/constants/permissions.js
? src/org/services/orgService.js
? supabase/functions/_shared/org.ts
? supabase/migrations/20260324160000_org_rls_policies.sql

Base model

Permission resolution is layered:

1. system defaults by role key
2. org role template permissions
3. member-specific overrides

This is why a member can inherit most permissions from a template but still have a local override for one capability or a custom numeric credit limit.

Important permission keys

The current org admin and workspace surfaces rely heavily on:

- ? can_publish
- ? publish_requires_final_approval
- ? can_schedule
- ? can_manage_library
- ? can_approve_library_uploads
- ? can_create_channels
- ? can_invite_members
- ? monthly_credit_limit

Important helper functions

The SQL and edge layers use helper functions so policies do not duplicate business logic everywhere.

The most important are:

- ? org_current_user_is_active_member(organization_id)
- ? org_current_user_role(organization_id)
- ? org_current_user_has_brand_access(organization_id, brand_project_id)
- ? get_member_permission(organization_id, permission_key)

These helper functions are central to the whole org admin story.

If they are wrong, every surface is wrong.

11. RLS and Security Model

The main org RLS file is:

- ? supabase/migrations/20260324160000_org_rls_policies.sql

Practical RLS summary

Members can read org-scoped data they belong to

This includes:

- ? organization
- ? membership
- ? brand projects they have access to
- ? role templates
- ? pipeline configs within allowed scope

Org owners/admins can mutate configuration data

This includes:

- ? organization metadata
- ? members
- ? role templates
- ? invitations
- ? pipeline configs
- ? some credit-review actions

Brand access matters

Many policies check both:

- ? active org membership
- ? brand-project access when the row is brand-scoped

That means an org member can be valid for the organization but still be denied from a brand-specific row if their `brand_project_ids` do not allow it.

Security boundary design

The repo uses a mixed model:

- ? some sensitive actions are handled through edge functions
- ? many admin-page writes still go directly through client-side Supabase calls and rely on RLS

This is workable, but it means new engineers must treat RLS as part of the application logic, not as a hidden database detail.

12. Org Bootstrap and Provisioning

This is one of the most important sections for recreating the system exactly.

How a new organization is created

The preferred path in the current repo is:

1. A platform admin opens `CreateOrgPanel`.
2. `src/admin/services/orgAdminService.js` creates the `organizations` row.
3. It invokes the `org-invite-member` edge function.
4. The edge function bootstraps the organization and sends the owner invitation.

What the bootstrap path creates

`ensureOrganizationBootstrap()` in `supabase/functions/_shared/org-bootstrap.ts` ensures the org has:

- ? owner membership
- ? default org role templates

- ? default brand project
- ? default pipeline configs
- ? baseline org setup needed for the workspace to function

Why this matters

A developer can manually insert rows, but that is not the safest way to recreate the system.

If you want the same system behavior the product expects, use the real bootstrap path.

13. Page-by-Page Deep Dive

13.1 Members Page

File:

? src/org/admin/MembersPage.jsx

What the page does

The page shows the org member directory and opens the invite/member editing tools.

Main capabilities

- ? list members
- ? search/filter visible members
- ? open invite panel
- ? assign role template
- ? edit brand-project access
- ? edit permission overrides
- ? change per-member credit limit override
- ? view last active state
- ? protect owner from accidental downgrade/removal logic in the UI

Data sources

- ? fetchOrganizationMembers(organizationId)
- ? fetchOrgRoleTemplates(organizationId)
- ? updateOrganizationMember(...)
- ? inviteOrganizationMember(...)

Invite flow

InviteMemberPanel.jsx lets an admin:

- ? choose email
- ? choose org role template

- ? optionally scope to specific brand projects
- ? preview permission implications before sending

This is much better than the older "pick a role from a generic select" pattern.

Important logic

- ? role selection updates inherited permissions
- ? brand scope can be all-brands or specific projects
- ? permission overrides are effectively tri-state:
 - ? inherit
 - ? allow
 - ? block

13.2 Roles Page

File:

```
? src/org/admin/RolesPage.jsx
```

What the page does

This is the template editor for org roles.

Main capabilities

- ? load system roles and custom roles
- ? create custom roles
- ? duplicate a role
- ? edit permission bundles
- ? delete custom roles
- ? block delete when members still use the role

Important logic

- ? system roles are protected differently from custom roles
- ? numeric permissions such as `monthly_credit_limit` are editable alongside boolean permissions
- ? effective member behavior depends on role template plus member override, not role template alone

13.3 Pipeline Config Page

File:

```
? src/org/admin/PipelineConfigPage.jsx
```

What the page does

This page is the workflow builder for review pipelines.

Main capabilities

- ? create pipeline configs
- ? duplicate configs
- ? delete configs
- ? set default config
- ? brand-scope a config or make it org-wide
- ? edit the stage array

What a stage can define

- ? stage name
- ? assignee role
- ? specific assignee user
- ? SLA / due-time style metadata
- ? escalation
- ? whether rejection comment is required
- ? whether the stage is optional
- ? whether client review links are involved

Why it matters

When a user submits a draft from My Office, `submitPostToPipeline()` uses this configuration to create a `pipeline_items` row and determine the review path.

13.4 Credit Management Page

File:

- ? `src/org/admin/CreditManagementPage.jsx`

What the page does

Shows the organization's credit situation and the list of credit requests.

Current maturity

This page is functional but thinner than the others.

It currently emphasizes:

- ? org monthly allocation
- ? org used credits
- ? pending request count

? request history / request rows

The repo also includes backend review capability via `credit-request-action`, but the current UI is not as fully action-heavy as the rest of the admin suite.

13.5 Org Settings Page

File:

? `src/org/admin/OrgSettingsPage.jsx`

What the page does

Displays the most basic org configuration summary.

Current maturity

This is intentionally minimal at the moment.

It exposes useful read-only context such as:

? org name

? plan key

? default pipeline reference from `organizations.settings`

It should be treated as a thin settings surface, not a full org configuration console.

14. Service Layer Deep Dive

`src/org/services/orgService.js`

Primary responsibilities:

? load org context

? load organization members

? load and mutate role templates

? update organization members

? invite organization members

Important public functions:

? `fetchOrganizationContext(...)`

? `fetchOrganizationMembers(...)`

? `fetchOrgRoleTemplates(...)`

? `createOrgRoleTemplate(...)`

? `updateOrgRoleTemplate(...)`

? `deleteOrgRoleTemplate(...)`

? `duplicateOrgRoleTemplate(...)`

? updateOrganizationMember(...)

? inviteOrganizationMember(...)

src/org/services/pipelineService.js

Primary responsibilities:

? manage pipeline configs

? provide pipeline template presets

? submit content into pipeline review

? load pipeline items

Important public functions:

? fetchPipelineConfigs(...)

? createPipelineConfig(...)

? updatePipelineConfig(...)

? deletePipelineConfig(...)

? duplicatePipelineConfig(...)

? setDefaultPipelineConfig(...)

? fetchPipelineItems(...)

? submitPostToPipeline(...)

src/org/services/creditService.js

Primary responsibilities:

? read credit requests

? create credit requests

? review credit requests

Important public functions:

? fetchCreditRequests(...)

? createCreditRequest(...)

? reviewCreditRequest(...)

15. User Flows

Flow 1: Create a new organization

1. Platform admin opens Create organization.
2. Enters org name, slug, owner email, and plan.
3. Frontend creates the org row.
4. org-invite-member is invoked with bootstrap_organization: true.

5. Bootstrap creates org owner membership, default role templates, default brand project, and default pipelines.
6. Owner receives invitation or password-setup path.
7. Owner logs in and can access `/app/org/:orgId/admin/*`.

Flow 2: Invite a member

1. Org admin opens Members.
2. Clicks Invite.
3. Selects the role template.
4. Chooses brand scope.
5. Reviews permission preview.
6. Sends invite.
7. `org_invitations` row is created and the email flow is triggered.
8. Accepted invite becomes an active member row.

Flow 3: Change a member's effective access

1. Org admin opens a member row.
2. Changes assigned role template.
3. Optionally narrows or broadens brand access.
4. Optionally overrides specific permissions or monthly credit limit.
5. Saves changes.
6. Effective access in the wider org workspace changes immediately because downstream pages depend on membership/permission helpers.

Flow 4: Change the review workflow

1. Org admin opens Pipelines.
2. Creates or edits a pipeline config.
3. Changes stages and assignments.
4. Sets it as default if required.
5. Future draft submissions from My Office use the updated config.

Flow 5: Review credits

1. Org admin opens Credits.
2. Sees org allocation and current usage.
3. Sees pending requests and historical requests.
4. Backend review actions exist, but the UI remains lighter here than the rest of the suite.

16. Screen Guide

No browser screenshots were generated from this CLI session, so this handoff includes detailed screen descriptions instead.

Members screen

+-----+-----+		
	Members header / summary	
	[Invite member]	
+-----+-----+		
	Member table/list	Selected member panel / drawer
	- name	- role template
	- email	- brand scope
	- role	- permission overrides
	- status	- credit override
	- last active	- save actions
+-----+-----+		

Roles screen

+-----+-----+		
	Role list	Role editor
	- system roles	- name / description
	- custom roles	- permission toggles
	[New role]	- monthly credit limit
	[Duplicate/Delete]	- save / delete
+-----+-----+		

Pipelines screen

+-----+-----+		
	Pipeline config list	Stage builder
	- default badge	- stage cards
	- org/brand scope	- assignee role or user
	[New / Duplicate]	- requirements / SLA / escalation
	[Set default]	- reorder / add / remove
+-----+-----+		

Credits screen

+-----+-----+		
	Credit summary cards	
	- monthly allocation	
	- used this period	
	- pending requests	
+-----+-----+		
	Credit request table	
	- requester	
	- amount	
	- reason	
	- status	
	- reviewed by / reviewed at	
+-----+-----+		

Settings screen

+-----+-----+		
	Organization summary	
	- name	
	- plan key	
	- default pipeline	
	- settings snapshot	
+-----+-----+		

17. Challenges, Corrections, and Design Decisions

1. Role and membership drift

The repo has historical overlap among:

- ? profiles.organization_id
- ? admin_roles.organization_id
- ? organization_members.organization_id

And for roles:

- ? auth metadata
- ? profiles.role
- ? admin_roles.role
- ? organization_members.role
- ? organization_members.org_role_key

This drift is a legacy burden from earlier admin iterations.

Correction path taken

- ? the org workspace increasingly relies on organization_members plus org_role_templates
- ? helper functions centralize permission checks
- ? the recommended forward path is to treat the org workspace model as canonical

2. RLS recursion

Early helper/policy combinations created recursive access evaluation issues.

Correction path taken

- ? 20260313090000_admin_rls_recursion_hotfix.sql
- ? helper logic moved into safer definers / cleaner access paths

3. Invite flow maturity

The original admin-style invite approach was too shallow for org work because it did not give enough visibility into downstream access.

Correction path taken

- ? org role templates became the invite basis
- ? invite panel now previews access implications
- ? brand scope is set during invite rather than cleaned up later

4. Credits UI is behind credits backend

The credits schema and backend actions are broader than what the UI currently exposes.

Current reality

- ? monitoring works
- ? request data works
- ? full admin action depth is still lighter than it could be

5. Settings page is intentionally thin

This is not a bug in the strict sense.

It reflects the current product maturity.

The control plane is strongest in members, roles, and pipelines.

6. Theme consistency

The org admin pages effectively became the design baseline for the org dashboard dark theme.

Later workspace surfaces were aligned back toward this darker org theme after separate exploratory visual passes.

18. Known Gaps and Risks

These are the most important issues for a new engineer to understand before making changes.

Structural / model risks

- ? There is still historical overlap between platform-admin governance data and org-workspace governance data.
- ? Some legacy compatibility fields remain in circulation.

UI maturity gaps

- ? Credits is not yet a full action console.
- ? Settings is still minimal.

Operational risks

- ? Some admin writes still rely on direct client-side database mutations plus RLS.
- ? If helper functions or policies drift, symptoms may appear across many org surfaces at once.

Audit-noted cleanup candidates

- ? clean up remaining policy drift in some older platform-admin tables
- ? reduce duplicate authority sources for roles and org scope
- ? keep tightening canonical ownership of membership and permissions

Small visible quality issues

Some org/admin files still contain text-encoding artifacts like " or €.

These are cosmetic, but they are worth cleaning because they make the product feel less finished and can confuse documentation reviewers.

19. Step-by-Step Rebuild / Setup Guide

This section is written so a new developer can stand up the same system with minimal guesswork.

Prerequisites

Install:

- ? Node.js
- ? npm
- ? Supabase CLI
- ? access to a Supabase project

Optional but useful:

- ? GitHub CLI if you use the repo deploy flow
- ? Deno/Supabase function tooling for local function testing

Step 1: Install dependencies

```
npm install
```

Step 2: Configure frontend environment variables

At minimum, the frontend needs:

```
VITE_SUPABASE_URL=...  
VITE_SUPABASE_ANON_KEY=...
```

Common optional vars seen in the repo:

```
VITE_APP_URL=...  
VITE_GROQ_API_KEY=...  
VITE_GROK_API_KEY=...  
VITE_LLM_PROVIDER=...  
VITE_GROQ_VISION_MODEL=...  
VITE_GROK_MODEL=...
```

For the org admin pages themselves, the only strictly required frontend vars are the Supabase URL and anon key.

The AI vars matter for wider workspace features.

Step 3: Configure Supabase / function secrets

Relevant secrets for org-admin provisioning and credit flows include:

```
SUPABASE_URL=...  
SUPABASE_ANON_KEY=...  
SUPABASE_SERVICE_ROLE_KEY=...  
APP_URL=...
```

Useful optional email-related secrets for the invitation flow:

```
RESEND_API_KEY=...  
FROM_EMAIL=...  
FROM_NAME=...
```

Step 4: Apply the required migrations

Minimum org-admin-related migration set:

```
supabase/migrations/20260312153000_admin_foundation.sql
supabase/migrations/20260313090000_admin_rls_recursion_hotfix.sql
supabase/migrations/20260324100000_org_workspace_foundation.sql
supabase/migrations/20260324110000_org_pipeline_tables.sql
supabase/migrations/20260324140000_org_credit_tables.sql
supabase/migrations/20260324160000_org_rls_policies.sql
supabase/migrations/20260324170000_org_helper_functions.sql
supabase/migrations/20260324180000_org_seed_plan_data.sql
supabase/migrations/20260324190000_org_invitation_owner_provisioning.sql
```

Recommended command:

```
supabase db push
```

If you want the full current org workspace, apply the later org workspace migrations too, not only the admin subset.

Step 5: Deploy the required edge functions

At minimum for the org-admin provisioning flow:

```
org-invite-member
credit-request-action
```

Deploy with the Supabase CLI as appropriate for your environment.

Step 6: Run the app

```
npm run dev
```

Step 7: Recreate the system using the real bootstrap path

Do not manually seed all org rows by hand unless you are debugging something specific.

Preferred path:

1. log in as a platform admin
2. open the platform admin organization creation flow
3. create an org with an owner email and plan
4. let the org-invite-member function bootstrap the org
5. accept the invite as the owner
6. navigate to `/app/org/:orgId/admin/members`

This produces the current system much more reliably than ad hoc manual inserts.

Step 8: Smoke-test the org admin surface

Minimum verification checklist:

1. Open `/app/org/:orgId/admin/members`.
2. Confirm members and role templates load.

3. Send a test invite.
4. Edit a member's brand-project access and permission overrides.
5. Open `/app/org/:orgId/admin/roles`.
6. Create a custom role and duplicate it.
7. Open `/app/org/:orgId/admin/pipelines`.
8. Create or edit a pipeline config and set a default.
9. Open `/app/org/:orgId/admin/credits`.
10. Confirm usage and request rows load.
11. Open `/app/org/:orgId/admin/settings`.
12. Confirm org metadata renders.

Step 9: Production-style verification

Run a build before handoff:

```
npm run build
```

20. Recreating the System Exactly

If the goal is not just "make it run" but "make it behave like this repo", follow these principles:

1. Use the real provisioning flow.
2. Use the current RLS policies and helper functions, not simplified mock rules.
3. Keep `organization_members` plus `org_role_templates` as the primary org-workspace access model.
4. Preserve the role-template plus member-override model.
5. Preserve brand-project scope as first-class data.
6. Preserve the pipeline config model because downstream workflow pages depend on it.

If you bypass any of those points, you may recreate the screens but not the same system behavior.

21. Best Practices for Future Engineers

Treat RLS as application logic

Do not treat the database policies as a black box.

If you change org permissions in the UI or services, review the matching RLS and helper functions.

Prefer canonical sources

When possible:

- ? use `organization_members` for org membership
- ? use `org_role_templates` for role definitions
- ? use helper functions for permission checks

Avoid creating new parallel sources of truth.

Be careful with compatibility fields

The repo still contains older admin compatibility paths.

Do not casually extend those unless you are intentionally working on backward-compatibility behavior.

Avoid bypassing the bootstrap path

If you are testing org setup, use the real create-org plus invite flow.

Most "mysterious missing data" bugs happen when bootstrap never ran.

Preserve the distinction between platform admin and org admin

Platform admin governs the platform.

Org admin governs one organization.

Do not blur those scopes.

Keep permissions explainable

When changing the model, make sure a future developer can still answer:

- ? What does this member inherit?
- ? What is locally overridden?
- ? What brands can they access?
- ? Why can or can't they perform a specific action?

Clean up visible string corruption

If you touch org admin files, remove mojibake and normalize copy to plain ASCII.

Small quality fixes matter in admin surfaces because admins rely heavily on clarity.

22. Recommended Reading Order for a New Developer

Best order:

1. this document
2. `src/router/router.jsx`
3. `src/utils/protectedRoute.jsx`
4. `src/layouts/OrgWorkspaceShell.jsx`
5. `src/org/context/OrgContextProvider.jsx`
6. `src/org/constants/permissions.js`
7. `src/org/services/orgService.js`
8. `src/org/admin/MembersPage.jsx`
9. `src/org/components/InviteMemberPanel.jsx`

10. src/org/admin/RolesPage.jsx
11. src/org/admin/PipelineConfigPage.jsx
12. src/org/services/pipelineService.js
13. src/org/admin/CreditManagementPage.jsx
14. src/org/services/creditService.js
15. src/org/admin/OrgSettingsPage.jsx
16. supabase/migrations/20260324160000_org_rls_policies.sql
17. supabase/migrations/20260324170000_org_helper_functions.sql
18. supabase/functions/org-invite-member/index.ts
19. supabase/functions/_shared/org-bootstrap.ts
20. the audit docs in docs/

23. Final Take

If someone new joins the project and asks, "What is the Org Admin Page really responsible for?" the honest answer is:

It is the governance layer of the org workspace.

It does not generate content, chat with the team, or schedule posts directly.

Instead, it defines the rules that let those things happen safely:

- ? who is in the org
- ? what they can do
- ? what scope they can work in
- ? what review flow content follows
- ? how credits are observed and governed

The strongest parts of the current implementation are:

- ? members
- ? roles
- ? pipelines
- ? org-scoped permissions and RLS

The weakest or thinnest parts are:

- ? credits UI depth
- ? settings depth
- ? remaining legacy model drift around older admin structures

As a handoff target, this is already a serious, usable control plane.

The next level of maturity is less about inventing new surfaces and more about tightening canonical ownership, reducing legacy drift, and deepening the thinner pages without breaking the permission model that already works.